Functions

Functions are self-contained chunks of code that perform a specific task. You give a function a name that identifies what it does. This name is then used to 'call' the functions to perform its task when needed.

When you define a function, you can choose to define one or more named and typed values that the function takes as input. These values are called parameters. You can also choose to define a type of value that the function will pass back as output when it's done. This value is called the function's return value.

We have been using functions this whole time that we have been programming. The most common one, and the easiest to identify, is the `print` function. `print` is the name of the function, and when we use the `()` parens we are calling the function. The text we put between the `()` parens is a parameter of the `print` function. `print` uses the value we pass to it to output that value into the console.

```
print("Printing is a function")
```

`print` also has some other parameters we can use if we want, or we can simply pass multiple, comma separated values for `print` to output on a single line.

```
print("foo", "bar", "baz")
print("foo", "bar", "baz", separator: "|", terminator: "/")
```

Now that we have talked a bit more about using functions, let's go ahead and write our first function.

Writing A Function

```
func helloWorld() {
    print("Hello World!")
}

// We then call our function by using the name of the function
// followed by `()` parens.
helloWorld()
```

Here we have a couple pieces of syntax that we use to declare a function, but the whole thing should at least feel a little familiar. We use the keyword `func` to say we are declaring a function. This is followed by the name of the function, we use lower camel case because

we are not creating a new type. Then we have parens, and then curly braces to show what code the function will run.

Declaring Parameters

We can also declare parameters that our function requires in order to run, this allows us to also access information from outside our function in a clean and easy way. Let's write a function to greet someone, in order to greet someone we need their name right? So we would make that a parameter of our function.

```
func hello(name: String) {
    print("Hello \(name)")
}

// This lets us call the same function, but with a different name
// each time!
hello(name: "Brock")
hello(name: "Misty")
```

Parameter Labels

Parameter Labels are a neat feature of Swift that I don't believe exists in any other language. They allow us to change how a function is called, by default when we are calling a function we have to put the names of the parameters before defining a value. So above we had the parameter named `name` and so we had to write that. However, it may not read very well like that, parameter labels let us fix that. They also make it so that we can ignore the name completely by using an `_` as the label.

```
func scheduleMeeting(with person: String, at time: Int) {
    print("Scheduled a meeeting with \(person), at \(time)")
}
```

Now that we have created the function with parameter labels, let's look at how they changed how we call our function. I am going to write down the proper way that we would call this function now, followed by how we would have done it without parameter labels.

```
scheduleMeeting(with: "Kassandra", at: 1230)
// scheduleMeeting(person: "Kassandra", time: 1230)
```

Comparing the two I imagine you can see how much easier our function is to read with thoughtful labels. You won't always need labels, but when you do, they make your code much more clear.

Default Values

So far we have covered a lot with functions, and there is still more to cover, so please stick with me. When we have parameters, there will be times when we may expect a certain value most of the time to be passed through, but we still want to be able to change it when that's not the case. In these situations where we expect a certain value a majority of the time, it can be tedious to have to insert that value every time we call the function. Luckily, Swift supports providing default values for parameters. This means that when we declare a parameter, we can choose to provide a value that will be used if when the function is called a different value is not provided. I think this is easier seen than explained, so let's do an example.

```
func greet(_ person: String, nicely: Bool = true) {
    if nicely {
        print("Hello \(person)!")
    } else {
        print("Oh no it's \(person) again...")
    }
}
```

I like to think of myself as a nice person, so by default I like to greet people nicely. However, every once in a while I'll meet someone who I am not too fond of, so I may want to not be as kind when I greet them. This is a great example of how we can use default values. Let's go ahead and try using this function to see how this default value changes how we call it.

```
greet("Doge")
greet("Cheems")
greet("Pepe", nicely: false)
```

Let's break down everything that we have done now that we have created and used our function. We have our function, with two parameters, one has a label `_`, and the name `person`, the other doesn't have a label, and has the name `nicely`. That stuff we have covered before, so now let's cover the rest. After our nicely parameters Type declaration, we put and `=` equal sign followed by the value `true`. This is our default value, or the value that will be used if no other value is provided for that parameter.

Now let's look at how we called the function, the first two function calls were `greet("Value")`, notice we didn't provide or even have to write anything in relation to the `nicely` parameter. We also didn't have any parameter labels or anything for the first parameter. This is because we used the `_` underscore label, which means that when we call the function, we don't have to put a label or name for that parameter. The third call is a little different though. The lack of a label or name for the first parameter is the same;

however, this time we put `nicely: false`. Put simply we provided a different value for the `nicely` parameter of our function. This means that instead of using the default value of `true` like we did with the other function calls, this time it is going to use whatever value we provided, so `false` in this example.

Returning Values

As I stated when we started this lecture, our functions can also return a value. I may be wrong in saying, but I would bet that you will write more functions that return things than ones that don't. SO PAY ATTENTION.

When we want a function to return a value, we add a bit of `Syntax` after the parameters but before the `{}` curly braces. We have to specify what `Type` of value we are returning so that Swift knows how to handle it. Let's write a function that will take a word as a parameter, and return the word reversed.

```
func reverse(word: String) -> String {
    var result = ""
    for char in word {
        result.insert(char, at: result.startIndex)
    }
    return result
}
```

So here we have two major new pieces of `Syntax` we have the `->` and the keyword `return`. We use the `->` after our parameters to specify that this function is going to return a value. Following the `->` we put what `Type` our function is going to return. Then we can start the body of our function using the `{}` curly braces. We also have this new `return` keyword. `return` is saying that we want to end our function and return the value we put after it. We can use return similarly to using `break` in loops. We can even use return in a function that doesn't return a value, we just wouldn't put a value after the `return` keyword.

Let's do an example showing how we can use `return` without a return value. We are going to rewrite one of the functions we already did.

```
func newGreeting(to person: String, nicely: Bool = true) {
    if nicely {
        print("Hello \(person)!")
        return
    }

    print("Oh no, it's \(person) again...")
}
```

```
newGreeting(to: "Jake")
newGreeting(to: "Ice King", nicely: false)
```

So this is very similar to our function we did before, but we didn't need an else statement, why? Since we put a `return` inside the `if statement` this means that when we hit that `return` our function will stop executing, so the second `print` wouldn't run.

When working with functions that can return a value, it is important to note that you must ALWAYS return a value from those functions. Meaning you cannot have any paths not return a value. So if you are writing code that has branches, like with if statements, you must always end up at a return statement.

```
func getGreeting(for person: String, isNice: Bool = true) -> String {
    if isNice {
        return "Hello \(person)!"
    } else {
        return "Oh no, it's \(person) again."
    }
}
```

If we were missing a return in either part of the `if-else` statement, we wouldn't be able to compile. It is very often to have a return as the last line at the root of your function if it returns a value.

Returning Complex Types

As I have said many times, when we are referring to types, we are including Simple Types, Collections, Enumerations, etc. Functions can return any of these types. They can also take any of these types as parameters. Let's make an example of a function that takes a number as a parameter, squares that number, and then returns the original number and the squared number in a tuple.

```
func squareMe(_ number: Int) -> (original: Int, squared: Int) {
    return (original: number, squared: number * number)
}

let squaredNumberInfo = squareMe(10)
print("The original number is \(squaredNumberInfo.original) and the
number squared is \(squaredNumberInfo.squared)")

// Use tuple decomposition
let (originalNumber, squaredNumber) = squareMe(5)
```

```
print("The original number is \(originalNumber) and the number squared
in \(squaredNumber)")
```

Variadic Parameters

Earlier we showed how `print` can take multiple values into a single parameter.

```
print("foo", "bar", "baz")
```

`print` can take any number of these values, when this is done it's called having `Variadic` parameters. It's super easy to implement in our own functions as well, let's do an example where we want to find the average of some `Doubles`.

```
// We specify a parameter is `Variadic` by putting `...` after the type.
func findAverage(of numbers: Double...) -> Double {
    var total = 0.0
    var count = 0.0
    // We treat variadic parameters as if they are an Array.
    for num in numbers {
        total += num
        count += 1
    }

    return total / count
}

let average = findAverage(of: 2, 12, 35)
print(average)
```

Implicit Return

Swift, as you guys have seen, offers a lot of really smart features that help us write less code. Functions also have a really nice feature. If your function is simple enough, you don't have to specify return in your function.

```
func isEven(_ number: Int) -> Bool {
    number % 2 == 0
}
```

In this example there is only one line in the function which is a `Boolean` expression. Since the value of that expression matches the return type of the function, the compiler assumes

that value of the expression is going to be returned. This allows us to omit the `return` part of the code.

Inout Parameters

In Swift, we can create what are called `inout` functions, what an `inout` function does is essentially update a variable you pass to it. Let's do some examples to show this off. Let's say that we have a function that takes a number, and then triples it.

```
func triple(_ num: Int) -> Int {
    num * 3
}

var number = 2
number = triple(number)
```

The above does let us modify a number with a new result, but it could be written in a cleaner way.

```
func tripleInPlace(_ num: inout Int) {
    num *= 3
}

tripleInPlace(&number)
print(number)
```

So now we have a function that can modify the contents of a variable directly. The syntax is a little different, so lets talk about it. There are two new pieces of syntax here: The `inout` keyword, and the `&` symbol. When we want an `inout` parameter we have to mark it with `inout` where we declare the type of the parameter. When we are providing a value into an `inout` parameter we have to preface it with an `&`. This tells the compiler that we are passing a reference to the variable as a whole, not a copy of the value it stores. Because we are passing a reference to the variable we cannot use `inout` parameters with constants, or values directly which is the main downside.